

Lab 7

This week covers topics from

- Missing and Outliers
- Naive Bayes

Submission And Assignment Instructions

Submit your `Lab7_lastname_firstname.pdf` file on canvas with your answers clearly marked (✅) and code commented. Skeleton code and/or comments are provided throughout to help you.

Formatting Notes:

- ❓ Questions you must answer or tasks are marked with a "❓"
- ✅ Answers you should give are marked with "✅ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ❓ YOUR CODE HERE`
 - ⭐ Bonus questions are marked with a "⭐"
 - 🍓 Bonus answers are marked with a "🍓"

Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1 - Bayes Rule

First we'll do some basic math with Bayes Rule:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)} \quad (1)$$

A new blood test has been developed for detecting early-stage breast cancer. The following data has been gathered:

- The prevalence of breast cancer in the high-risk population being screened is 2% ($P(C) = 0.02$)
- The test has a sensitivity of 92%, meaning it correctly identifies 92% of people who have cancer ($P(T|C) = 0.92$)
- The test has a specificity of 85%, meaning it correctly identifies 85% of people who don't have cancer ($P(T'|C') = 0.85$)

*The ' is sometimes used to denote the complement of a set in probability.
 $P(T)$ is the probability of a positive test, so $P(T')$ is the probability of a*

negative test. $P(T) = 1 - P(T')$. This also applies to conditional probabilities! $P(T|C) = 1 - P(T'|C)$

- Hint: Keep in mind that $P(T)$ can be calculated using the law of total probability:

$$P(T) = P(T|C)P(C) + P(T|C')P(C')$$

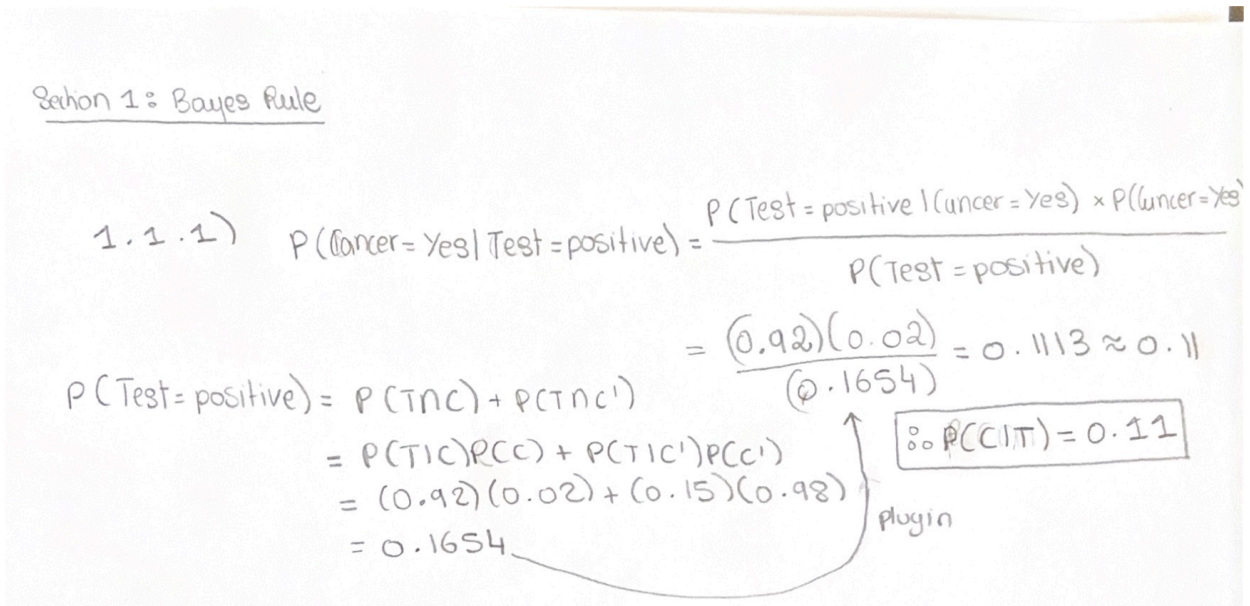
? 1.1.1 Bayes Rule

Using Bayes' rule, calculate the probability that a person actually has breast cancer given that they tested positive ($P(C|T)$).

Show your work and express your final answer as a probability rounded to two decimal places.

In [9]: # Answer:

```
from IPython.display import Image, display
display(Image("1.1.1.png"))
```



Section 1: Bayes Rule

1.1.1) $P(\text{Cancer} = \text{Yes} | \text{Test} = \text{positive}) = \frac{P(\text{Test} = \text{positive} | \text{Cancer} = \text{Yes}) \times P(\text{Cancer} = \text{Yes})}{P(\text{Test} = \text{positive})}$

$P(\text{Test} = \text{positive}) = P(T|C) + P(T|C')$

$= P(T|C)P(C) + P(T|C')P(C')$

$= (0.92)(0.02) + (0.15)(0.98)$

$= 0.1654$

$\therefore P(C|T) = 0.11$

plugin

? 1.1.2 In your own words, briefly explain (1 sentence) what this result means.

 Answer: Out of all the people who tested positive, how many of them actually had cancer. We found that 11% of the people who tested positive had cancer.

Section 1.2: Implement Naive Bayes Classifier

We will now implement a Naive Bayes classifier using scikit-learn:

A research team is developing a diagnostic tool for classifying potential breast cancer tumors as benign or malignant based on various clinical features. This is the same dataset from scikit-learn we have previously used with 30 features!

? 1.2.1 First load in the breast cancer dataset from scikit-learn. Then create a training (80%) and testing (20%) split. Be sure to scale the features using StandardScaler. Use 'random_state = 42' throughout.

Hint: Remember we don't fit the entire dataset when we scale

```
In [10]: import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt

# Load the breast cancer dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
)

# Scale the features
scaler = StandardScaler()

# Fit ONLY on training data
X_train_scaled = scaler.fit_transform(X_train)

# Use same transformation on test data
X_test_scaled = scaler.transform(X_test)
```

? 1.2.2 Next, initialize a Gaussian Naive Bayes classifier using scikit-learn. We then want to fit the training data and predict the testing data.

```
In [11]: # Train Naive Bayes model on training data
model = GaussianNB()
model.fit(X_train_scaled, y_train)
# Make predictions on testing data
y_pred = model.predict(X_test_scaled)
```

? 1.2.3 Finally, calculate and print the accuracy score and classification report metrics provided by scikit-learn. Classification report should include the precision, recall, and f1-score at a minimum.

```
In [12]: # Calculate and print metrics
acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)

print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Accuracy: 0.9649122807017544

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

Confusion Matrix:

```
[[40  3]
 [ 1 70]]
```

Section 2 - Missing Data

We are now going to use the same breast cancer dataset to investigate missing data using Isolation Forests. Before we begin, we will cover some conceptual content on when missing data methods can be used.

Section 2.1- Conceptual Missing Data

? 2.1.1 For each of the following missing data patterns, briefly explain whether listwise deletion would be appropriate and why:

- Missing Completely at Random (MCAR)
- Missing Not at Random (MNAR)

✓ Answer: MCAR: Listwise deletion is appropriate because the missingness is completely random and unrelated to the data values. Removing those cases does not introduce systematic bias, though it may reduce sample size.

MNAR: Listwise deletion is not appropriate because the missingness depends on the unobserved values themselves. Removing those cases would introduce bias and distort the data distribution.

? 2.1.2 Name one potential problem that can arise from using mean imputation for missing values.

✓ Answer: Mean imputation reduces variability in the data and can distort the true distribution, potentially biasing model estimates.

Section 2.2- Isolation Forest

In this section, we are going to once again be working with the breast cancer dataset. Instead of performing a classification task, we are now going to be identifying outliers using an Isolation Forest.

? 2.2.1 To start, load in the breast cancer dataset again and scale the dataset.

Hint: Since we are trying to identify outliers and no longer have a train/test split, we want to scale the entire dataset

```
In [13]: import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt
import seaborn as sns

# Load the breast cancer dataset
data1 = load_breast_cancer()
X1 = data1.data

# Scale the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X1)
```

? 2.2.2 Use the scikit-learn IsolationForest to define a forest with contamination=0.1 and random_state=42. We are now going to try 2 different methods to visualize the outliers.

First use fit_predict to get the labels of the outliers. Simply call fit_predict and pass in the scaled data. Next, use score_samples to get outlier scores through a decision function. Call this function and pass the dataset into the IsolationForest. This will return negative values for datapoints where a more negative number represents a point more likely to be an outlier.

Be sure to name the fit_predict output 'outlier_labels' and the score_samples output 'outlier_scores' so the next part of the code will run.

Contamination is the expected proportion of outliers

```
In [14]: # Apply Isolation Forest
model1 = IsolationForest(
    contamination = 0.1,
    random_state = 42)
# Fit and predict
# -1 for outliers, 1 for inliers
outlier_labels = model1.fit_predict(X_scaled)
# Get outlier scores (decision function)
# More negative = more likely to be outlier
outlier_scores = model1.score_samples(X_scaled)
```

? 2.2.3 Run this code to plot a histogram and scatter plot of outliers. Make sure your variable names align with the previous question and make sure your dataframe for the dataset is called df_scaled (or change the name of the dataframe).

```

In [15]: # Create visualization of outliers in first two features
df = pd.DataFrame(data1.data, columns=data1.feature_names)
df_scaled = pd.DataFrame(X_scaled, columns=data1.feature_names)

plt.figure(figsize=(10, 6))
plt.scatter(df_scaled.iloc[outlier_labels == 1, 0],
            df_scaled.iloc[outlier_labels == 1, 1],
            c='blue', label='Normal')
plt.scatter(df_scaled.iloc[outlier_labels == -1, 0],
            df_scaled.iloc[outlier_labels == -1, 1],
            c='red', label='Outlier')
plt.xlabel(df.columns[0])
plt.ylabel(df.columns[1])
plt.title('Outlier Detection using Isolation Forest')
plt.legend()
plt.show()

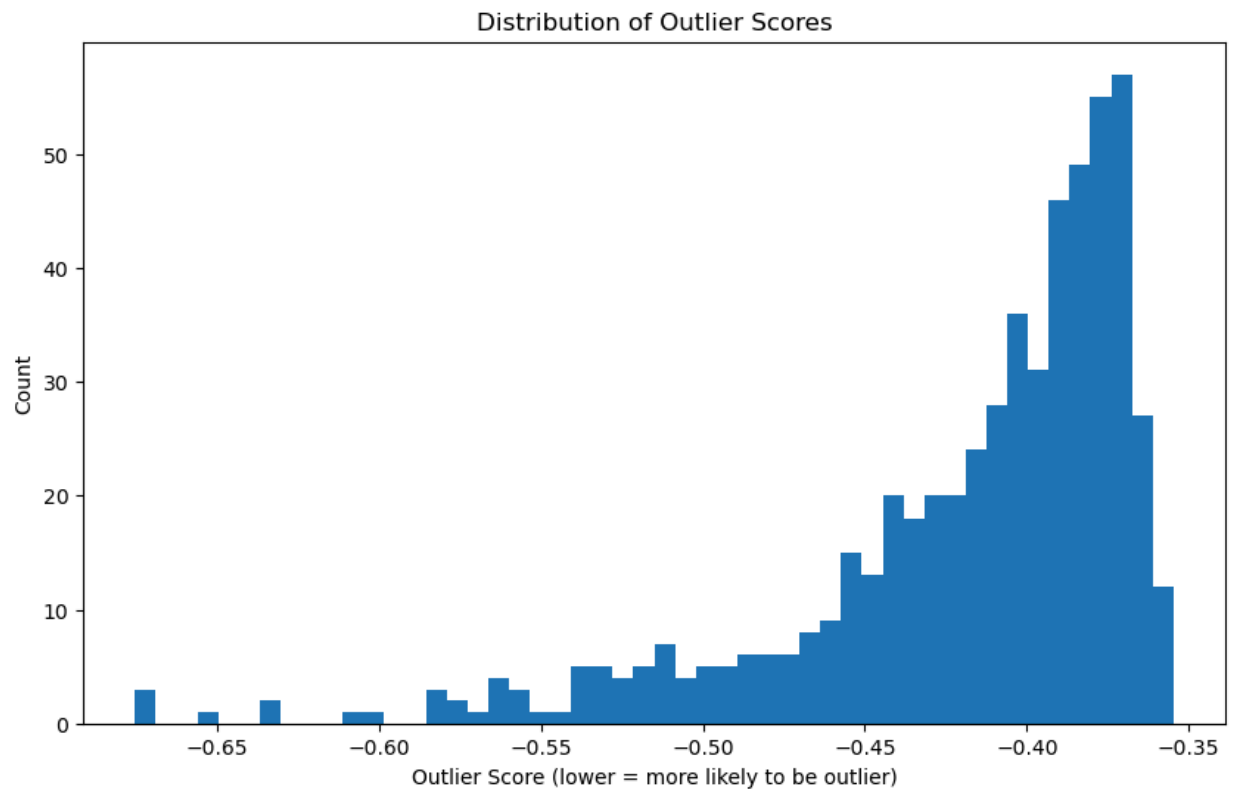
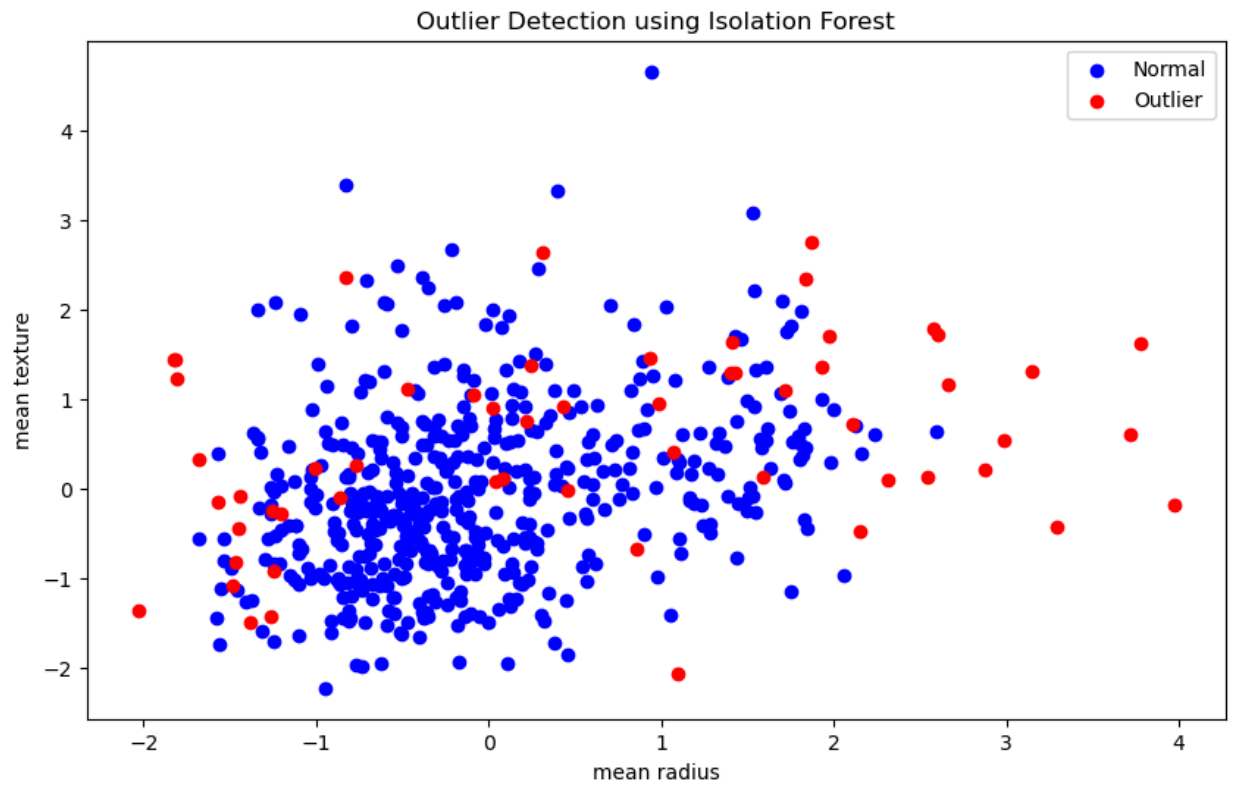
# Plot distribution of outlier scores
plt.figure(figsize=(10, 6))
plt.hist(outlier_scores, bins=50)
plt.title('Distribution of Outlier Scores')
plt.xlabel('Outlier Score (lower = more likely to be outlier)')
plt.ylabel('Count')
plt.show()

# Print summary statistics
n_outliers = sum(outlier_labels == -1)
n_total = len(outlier_labels)
print(f"\nOutlier Detection Summary:")
print(f"Total samples: {n_total}")
print(f"Number of outliers detected: {n_outliers}")
print(f"Percentage outliers: {100 * n_outliers/n_total:.1f}%")

# Show outlier scores for most extreme cases
outlier_df = pd.DataFrame({
    'Score': outlier_scores,
    'Is_Outlier': outlier_labels == -1
})
print("\nMost extreme outlier scores:")
print(outlier_df.nsmallest(57, 'Score'))

#notes for myself
#each tree randomly splits accross the features
#for each patient -> How many splits did it take before this point became isolated
#the longer a path = normal point / shorter path = outlier
#those scores are converted -> more negative, more abnormal
# contamination = 0.1 -> Label the 10% most abnormal points as outliers

```



Outlier Detection Summary:

Total samples: 569

Number of outliers detected: 57

Percentage outliers: 10.0%

Most extreme outlier scores:

	Score	Is_Outlier
461	-0.675419	True
122	-0.672363	True
212	-0.669896	True
152	-0.653559	True
108	-0.634180	True
3	-0.632376	True
12	-0.610638	True
78	-0.603528	True
82	-0.584586	True
258	-0.579900	True
190	-0.579351	True
68	-0.577908	True
352	-0.574541	True
9	-0.568588	True
42	-0.564215	True
567	-0.563902	True
71	-0.563900	True
0	-0.560320	True
505	-0.557130	True
213	-0.554469	True
265	-0.554033	True
180	-0.551855	True
504	-0.544422	True
290	-0.538197	True
368	-0.537388	True
202	-0.535158	True
379	-0.534866	True
314	-0.534592	True
181	-0.534198	True
568	-0.532563	True
112	-0.529881	True
351	-0.528827	True
376	-0.528087	True
192	-0.527224	True
25	-0.525771	True
503	-0.524455	True
151	-0.523651	True
176	-0.517330	True
562	-0.516340	True
101	-0.516276	True
538	-0.515869	True
561	-0.515564	True
468	-0.513919	True
521	-0.513813	True
400	-0.513618	True
83	-0.512309	True
563	-0.511801	True
252	-0.511689	True
539	-0.509982	True
236	-0.508383	True
318	-0.508070	True
116	-0.507949	True
38	-0.502501	True

339	-0.502006	True
564	-0.498913	True
203	-0.498340	True
430	-0.497832	True

Note: The outlier detection is performed in the full feature space, though visualization is limited to 2D for clarity.